

String Algorithms — Pattern Recognition & Universal Notes

These notes are meant for quick revision before solving string problems. They focus on recognizing common string patterns, traversal techniques, and reusable templates frequently used in coding interviews and contests.

1. How to Recognize a String Problem

Typical signals:

- "Substring"
- "Anagram"
- "Palindrome"
- "Pattern matching"
- "Character frequency"
- "Transform string"

Many string problems combine:

- hashing / frequency counting
 - two pointers
 - sliding window
 - stack
-

2. Basic String Traversal

Template:

```
for (int i = 0; i < s.size(); i++) {  
    char c = s[i];  
}
```

Time complexity:

$O(n)$

3. Character Frequency Pattern

Used in problems involving **anagrams** or **counting characters**.

Template:

```
vector<int> freq(26,0);

for(char c : s)
    freq[c-'a']++;
```

Used in:

- Valid anagram
 - Permutation in string
 - Ransom note
-

4. Palindrome Pattern

A string is palindrome if it reads same forward and backward.

Two pointer template:

```
int l = 0;
int r = s.size()-1;

while(l < r){

    if(s[l] != s[r])
        return false;

    l++;
    r--;
}

return true;
```

Used in:

- Valid palindrome
- Palindrome substring

5. Sliding Window Pattern

Very common in substring problems.

Idea:

Maintain window `[left ... right]`.

Template:

```
int left = 0;

for(int right = 0; right < n; right++){

    // expand window

    while(window invalid)
        left++;

    // update answer
}
```

Used in:

- Longest substring without repeating characters
- Minimum window substring
- Anagram substrings

6. Hash Map String Pattern

Used when characters are not limited to 26 letters.

Template:

```
unordered_map<char, int> mp;

for(char c : s)
    mp[c]++;
```

Used in:

- character frequency
 - substring counting
-

7. String Building Pattern

Sometimes we need to construct a new string.

Template:

```
string ans="";  
  
for(char c : s)  
    ans += transform(c);
```

Used in:

- string compression
 - reversing words
-

8. Stack with Strings

Used when characters must cancel or match.

Template:

```
stack<char> st;  
  
for(char c : s){  
    if(condition)  
        st.pop();  
    else  
        st.push(c);  
}
```

Used in:

- remove adjacent duplicates

- valid parentheses
-

9. KMP / Pattern Matching Idea

Used when searching pattern in string.

Basic idea:

Avoid rechecking characters by using prefix information.

Typical problems:

- find substring
 - repeated pattern detection
-

10. Mental Checklist Before Coding

Ask yourself:

1. Is this substring or subsequence?
2. Can sliding window work?
3. Do I need character frequency?
4. Is this palindrome check?
5. Do I need hashing or stack?

Once the pattern is identified, coding becomes mechanical.

11. Pattern Recognition Summary

Most string problems fall into these categories:

Pattern	Example
Frequency counting	Valid anagram
Palindrome check	Valid palindrome
Sliding window	Longest substring
Stack processing	Remove duplicates
String building	Reverse words

Pattern	Example
Pattern matching	KMP

Recognizing the pattern quickly is the key.

12. Universal String Templates

Traversal

```
for(char c : s)
```

Frequency array

```
freq[c-'a']
```

Two pointer palindrome

```
while(l < r)
```

Sliding window

```
for(right)
    while(invalid)
        left++
```

These templates solve the majority of string problems.

13. Important C++ String Methods

Common operations available in `std::string`.

Length

```
s.length();
s.size();
```

Access character

```
s[i]  
s.at(i)
```

Append

```
s += "abc";  
s.append("abc");
```

Push and pop character

```
s.push_back('a');  
s.pop_back();
```

Substring

```
s.substr(start, length);
```

Find substring

```
s.find("abc");
```

Returns index or `string::npos` if not found.

Replace

```
s.replace(pos, len, "new");
```

Insert

```
s.insert(pos, "abc");
```

Erase

```
s.erase(pos, len);
```

Clear string

```
s.clear();
```

Check empty

```
s.empty();
```

14. Useful STL Algorithms for Strings

Reverse string

```
reverse(s.begin(), s.end());
```

Sort characters

```
sort(s.begin(), s.end());
```

Count character

```
count(s.begin(), s.end(), 'a');
```

Transform case

```
transform(s.begin(), s.end(), s.begin(), ::tolower);  
transform(s.begin(), s.end(), s.begin(), ::toupper);
```

Remove character

```
s.erase(remove(s.begin(), s.end(), 'a'), s.end());
```

15. String to Number Conversion

String to integer

```
stoi(s);
```

String to long long

```
stoll(s);
```

String to double

```
stod(s);
```

Number to string

```
to_string(x);
```

16. Character Utility Functions

From `<cctype>` library.

Check character type

```
isdigit(c);  
isalpha(c);  
isalnum(c);  
isspace(c);
```

Case conversion

```
tolower(c);  
toupper(c);
```

17. Quick String Toolkit Summary

Most problems use combinations of these tools:

Tool	Use
freq array	anagrams / counting
sliding window	substring problems
two pointers	palindrome
stack	bracket / cancel problems
hashing	character frequency
substr/find	substring operations
reverse/sort	transformations

Master these utilities and string problems become much easier to implement.